Project: Fractal Forest Designer

1. Output (via console, GUI, or say block):

* **GUI (Primary):** The main output will be the visually drawn trees on the Snap! stage.

* **say block:**

* To prompt the user for inputs (e.g., "How many trees would you like to plant in your forest?").

* To give feedback (e.g., "Generating your forest...", "Forest Complete!").

* To display current settings if the user wants to review them.

* **(No console output needed unless for advanced debugging, say and GUI are better for Snap!)**

2. User Input:

* Use **ask [] and wait** blocks for the following:

* **Number of trees:** How many trees to generate in the forest.

* **Base size range:** Perhaps ask for a minimum and maximum base size for trees to add variety.

* **Recursion levels range:** Min/max levels for variety.

* **Color Scheme:**

* Option 1: Ask for trunk color and leaf color separately.

* Option 2: Offer a predefined list of schemes (e.g., "Spring", "Autumn", "Winter", "Fantasy").

* **Pen Thickness:** Base thickness for trunks.

* **Tree "Style" Choice:** (See custom block section)

* **(Optional) Tree Placement:** Ask if trees should be randomly placed or if the user wants to click to plant each one.

3. Repetition Structure (loop):

* A **repeat until <>** or **repeat ()** loop will be used to:

* Draw the specified **number of trees**. Inside this loop, you'd set the parameters (size, levels, color etc., possibly randomized within user-defined ranges) for *each* tree before calling your main recursive tree drawing block.

* If offering predefined color schemes or styles, you might loop through a list to present choices.

4. Branching Structure (if/then):

* **Inside your main recursive tree block:** This is already there for the base case (**if levels > 0 then...**).

* **For Color Scheme: if chosen_scheme = "Autumn" then set trunk_color to [brown] set leaf_color_palette to [list_of_autumn_leaf_colors] else if chosen_scheme = "Spring" then ...**

* **For Style Choice: if chosen_style = "Weeping" then set angle_modifier to [-10] else if chosen_style = "Spiky" then set angle_modifier to [5] ...**

* **For Pen Thickness (Tapering): if current_level > (total_levels / 2) then set pen size to (base_thickness / 2) else set pen size to base_thickness**. (This is a simple example; it could be more gradual).

* **Input Validation: if number_of_trees < 1 then ask "Please enter at least 1 tree" and wait**.

5. At least one original custom function/block per team member:

(we'll need at least one significant custom block. our recursive **tree** *is a great start. Let's define it more formally and add another one.)*

---

* **Block 1: `Draw_Recursive_Tree (levels, base_size, trunk_color, leaf_color_palette, angle_base, length_reduction_factor, current_pen_thickness, style_variant)`**
  * This is your enhanced main recursive tree block.
  * It takes more parameters for full customization.
  * `leaf_color_palette` could be a list of colors, and for the "leaf" levels, it picks one randomly.
  * `style_variant` could subtly alter angles or branch numbers.
  * It would handle tapering pen thickness internally.

* **Block 2: `Get_User_Forest_Preferences ()` (Reporter Block)**
  * **Purpose:** This block would contain all the `ask [] and wait` prompts for the user (number of trees, size range, level range, color scheme choice, style choice, base pen thickness).
  * **Output:** It would *report* a list containing all these user preferences. For example: `[[num_trees], [min_size, max_size], [min_levels, max_levels], [chosen_color_scheme_name], [chosen_style_name], [base_pen_thickness]]`.
  * This keeps your main script cleaner.

* **(Optional Extra Block Idea): `Apply_Tree_Style (style_name, current_level)` (Reporter Block)**
  * Takes a style name and the current recursion level.
  * Reports back specific drawing parameters (e.g., a list of `[branch_angle, num_branches_this_level, length_factor_modifier]`) based on the style. This allows for more distinct tree appearances.

---

6. At least one of: list, GUI, or file input or output:

* **List (Primary choice here):**
  * The output of `Get_User_Forest_Preferences ()` will be a list.

    *   `leaf_color_palette` inside `Draw_Recursive_Tree` would be a list of colors to choose from for the leaves.
    *   You could have a list of predefined "style" names for the user to choose from.
    *   A list of `[x, y]` coordinates for where trees have been planted (if you implement user-click placement).
*   **GUI (Already covered):** The visual drawing on the stage is GUI output. User input via `ask` is a form of simple GUI interaction.
    *   **(File I/O is more advanced for a typical Snap! project of this scope, focus on Lists and GUI).**

Conceptual Main Script Flow

```
when green flag clicked
    clear
    setup_initial_sprite_state // (e.g., go to bottom center, point up, pen down)

    // Get all settings from the user
    set [forest_settings v] to (Get_User_Forest_Preferences) // Calls your custom block

    // Extract settings from the list (or access them directly by list item index)
    set [num_trees_to_draw v] to (item (1 v) of (forest_settings))
    set [min_base_size v] to (item (1 v) of (item (2 v) of (forest_settings)))
    // ... and so on for all settings

    say (join "Generating " (join (num_trees_to_draw) " trees...")) for (2) secs

    repeat (num_trees_to_draw)
        // For each tree, potentially randomize its specific parameters based on user's range choices
        set [current_tree_size v] to (pick random (min_base_size) to (max_base_size))
        set [current_tree_levels v] to (pick random (min_levels) to (max_levels))
        // ... determine current colors, style variants etc.

        // (Optional) Position the tree
        pen up
        go to x: (pick random (-200) to (200)) y: (pick random (-150) to (0)) // Example random placement
        pen down

        // Call your main drawing block
        Draw_Recursive_Tree levels: (current_tree_levels)
                            base_size: (current_tree_size)
                            trunk_color: (chosen_trunk_color) // from settings
                            leaf_color_palette: (chosen_leaf_palette) // from settings
                            angle_base: (15) // or make this customizable
                            length_reduction_factor: (0.75) // or make this customizable
                            current_pen_thickness: (base_pen_thickness) // from settings
                            style_variant: (chosen_style_name) // from settings
```

```
end

say "Forest Complete!"
```

Question 1: Problem

Provide one or two well-written sentences summarizing the problem your program hopes to address.

Answer:

Creating visually interesting and varied fractal tree designs manually is time-consuming and limits artistic exploration. This program aims to provide users with an intuitive tool to easily generate diverse, algorithmically-drawn forests by specifying simple parameters.

Question 2: Overall Description

Provide several well-written sentences describing what your program will be.

Answer:

The "Fractal Forest Designer" will be an interactive Snap! program that allows users to generate entire forests of customizable fractal trees. Users will be prompted for various parameters such as the number of trees, their size range, recursion depth (branching levels), color schemes for trunks and leaves, and initial pen thickness. The program will then algorithmically render these trees on the stage, providing a unique visual output for each set of preferences, effectively enabling users to "grow" their own digital art.

Question 3: User Characteristics

Provide several well-written sentences describing the intended user. Include any pertinent information such as ages, abilities, interests, etc.

Answer:

The intended users are individuals interested in generative art, fractal patterns, or creative coding, potentially including students learning about recursion and parametric design in a visual context. Users could range from ages 10 and up, possessing basic computer literacy which includes the ability to read on-screen prompts and provide text or numeric input via a keyboard. No prior programming experience is required to use the program, making it accessible for those curious about algorithmic art generation or simply wanting to create unique tree visuals for fun or for simple projects.

Question 4: System Features

Provide several well-written sentences describing the various features of your intended program.

Answer:

The "Fractal Forest Designer" will feature user-selectable parameters for comprehensive forest generation. Key features include the ability for users to specify the total number of trees in the forest, a range for their initial base size, and a range for their recursion depth, which controls the complexity of branching. Users will also be able to define distinct colors for tree trunks and select or input a palette of colors for the leaves, alongside setting an initial pen thickness for drawing. To enhance visual appeal, trees will be drawn with randomized placement on the stage, and all user prompts for input will be both written on screen and audible using text-to-speech functionality for better accessibility and user experience.

Question 5: System Requirements

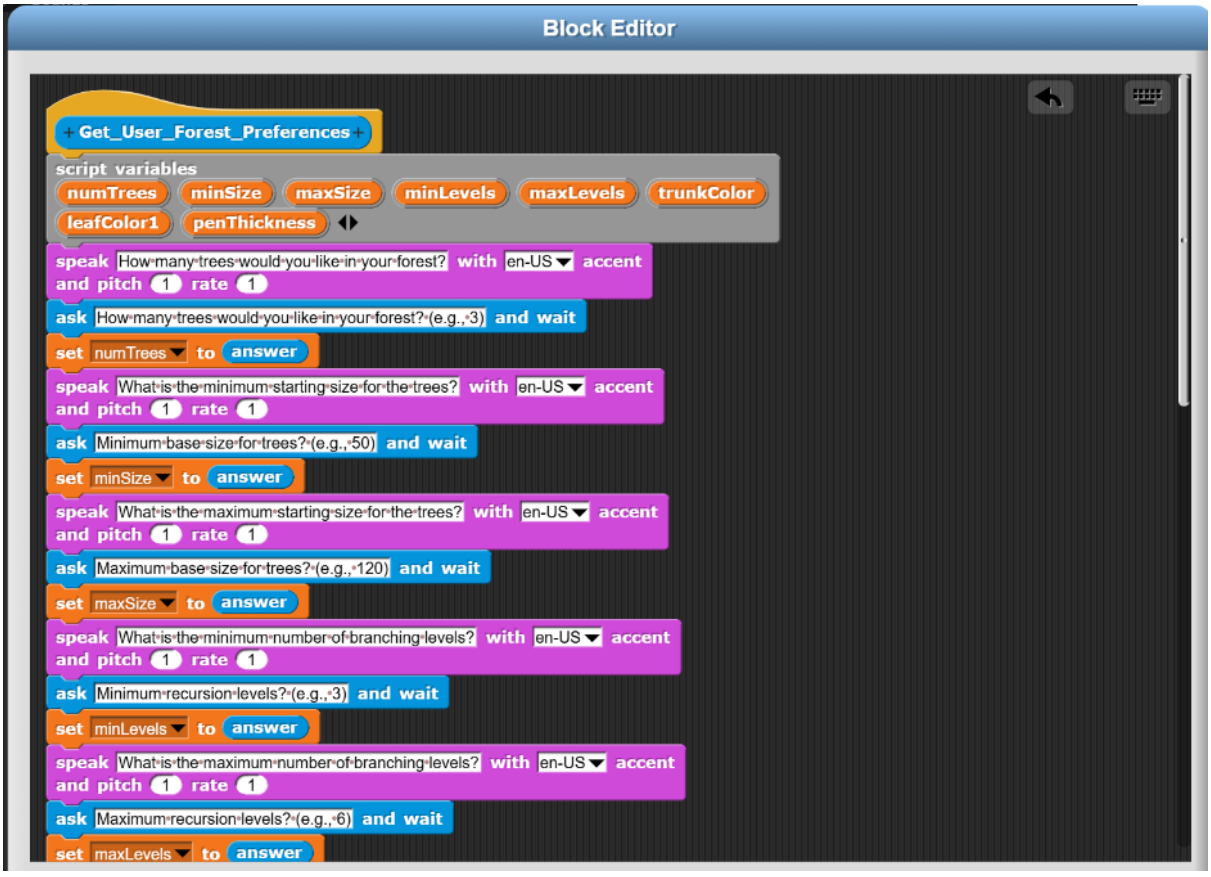
Please state whether your project will be in Snap! or Python.

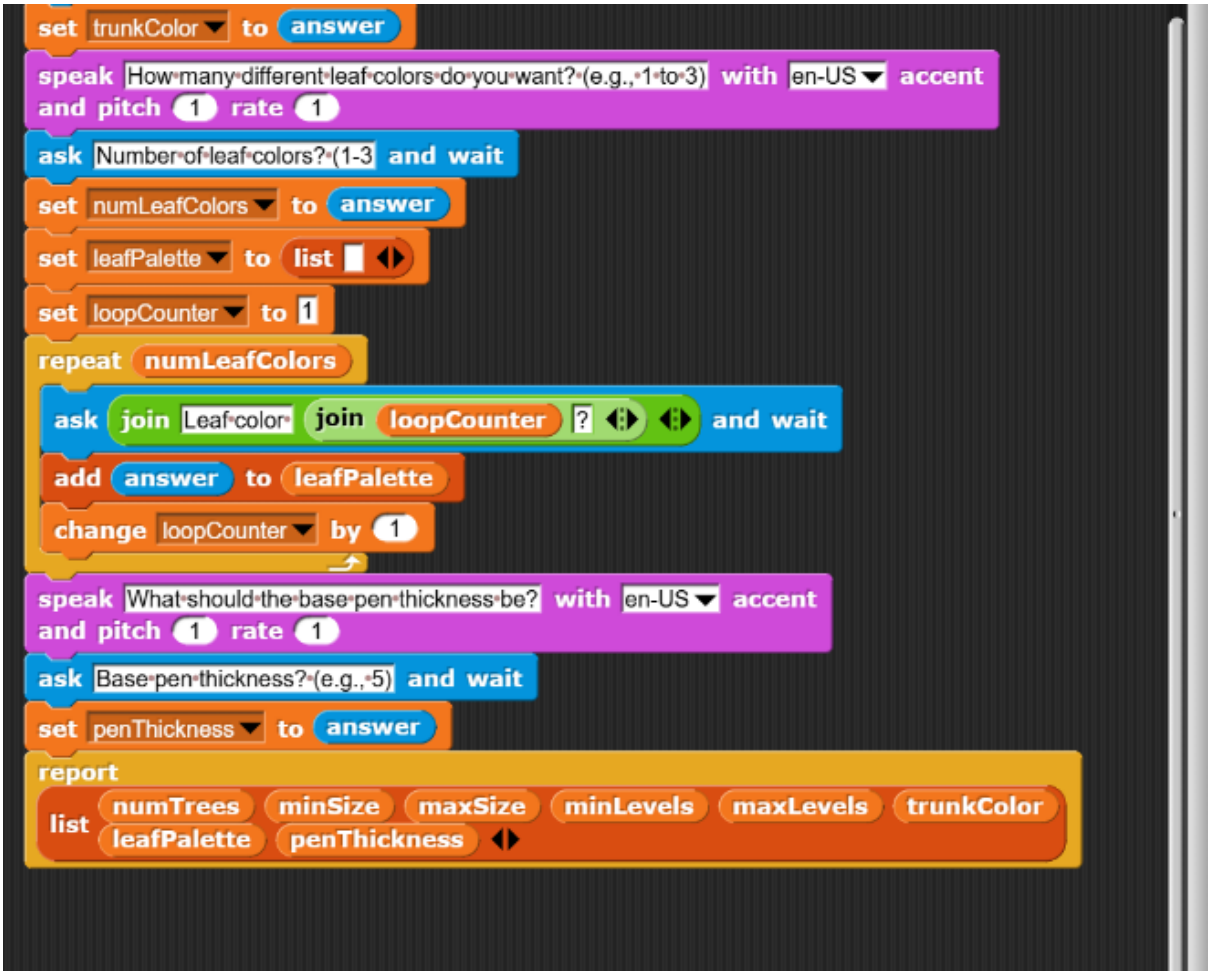
Answer:

This project will be developed entirely within the Snap! programming environment. To run the program, users will require a modern web browser with internet access that meets the current system requirements for Snap!, as detailed on the Snap! website (snap.berkeley.edu/requirements).

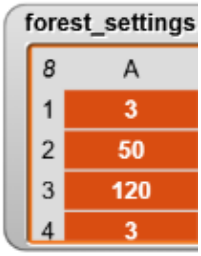
Input Interface:





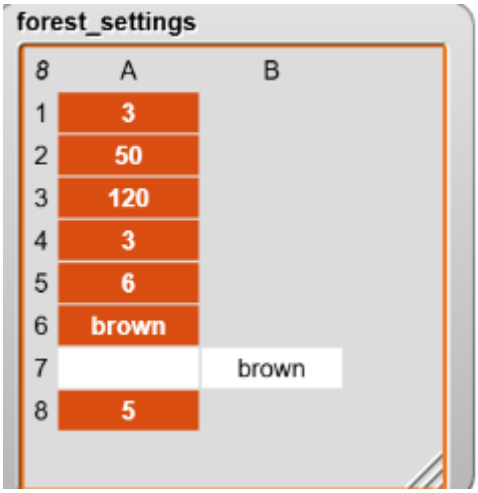

set trunkColor to answer
speak How many different leaf colors do you want? (e.g., 1 to 3) with en-US accent and pitch 1 rate 1
ask Number of leaf colors? (1-3 and wait
set numLeafColors to answer
set leafPalette to list
set loopCounter to 1
repeat numLeafColors
ask join Leaf color join loopCounter ? and wait
add answer to leafPalette
change loopCounter by 1
speak What should the base pen thickness be? with en-US accent and pitch 1 rate 1
ask Base pen thickness? (e.g., 5) and wait
set penThickness to answer
report
list numTrees minSize maxSize minLevels maxLevels trunkColor leafPalette penThickness







We save the results at list called forest_setting.




```
define Draw_Recursive_Tree (levels) (current_size) (trunk_color) (leaf_color_palette) (angle_base) (length_reduction_factor) (current_pen_thickness)

  // Only proceed if there are levels left to draw
  if <(levels) > (0)> then

    // --- Set properties for the current segment ---
    // 1. Set Pen Thickness
    set pen size to (current_pen_thickness)

    // 2. Set Pen Color (Distinguish between trunk/branches and leaves)
    //    We'll say anything with 'levels > 1' is trunk/branch, 'levels = 1' is a leaf.
    if <(levels) > (1)> then
      set pen color to (trunk_color)
    else // This is a leaf level (levels = 1)
      // Check if the leaf_color_palette has any colors
      if < (length of (leaf_color_palette)) > (0) > then
        set pen color to (item (pick random (1) to (length of (leaf_color_palette))) of (leaf_color_palette))
      else
        set pen color to [green] // Default leaf color if palette is empty or not provided
      end
    end

    // --- Draw the current segment ---
    move (current_size) steps

    // --- Prepare for and draw the LEFT sub-tree ---
    turn ↺ (angle_base) degrees

    // Recursive call for the LEFT branch
```

```
    Draw_Recursive_Tree levels: ((levels) - (1))
                current_size: ((current_size) * (length_reduction_factor))
                trunk_color: (trunk_color) // Pass along
                leaf_color_palette: (leaf_color_palette) // Pass along
                angle_base: (angle_base) // Pass along
                length_reduction_factor: (length_reduction_factor) // Pass along
                current_pen_thickness: ((current_pen_thickness) * (0.9)) // Example: Taper
thickness slightly (0.9 means 10% thinner)

    // --- Prepare for and draw the RIGHT sub-tree ---
    // Turn from the left branch orientation to the right branch orientation
    // (angle_base to get back to center + angle_base to turn right)
    turn ↻ ((angle_base) * (2)) degrees

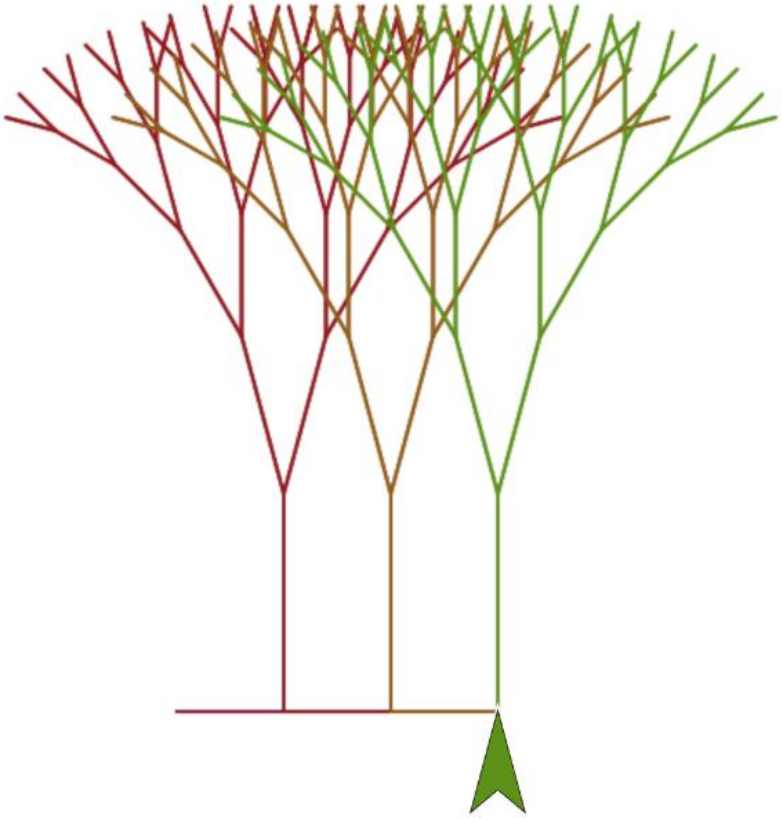
    // Recursive call for the RIGHT branch
    Draw_Recursive_Tree levels: ((levels) - (1))
                current_size: ((current_size) * (length_reduction_factor))
                trunk_color: (trunk_color) // Pass along
                leaf_color_palette: (leaf_color_palette) // Pass along
                angle_base: (angle_base) // Pass along
                length_reduction_factor: (length_reduction_factor) // Pass along
                current_pen_thickness: ((current_pen_thickness) * (0.9)) // Example: Taper
thickness slightly

    // --- Return sprite to original orientation for this segment ---
    // Turn from the right branch orientation back to the original upward orientation
    turn ↺ (angle_base) degrees

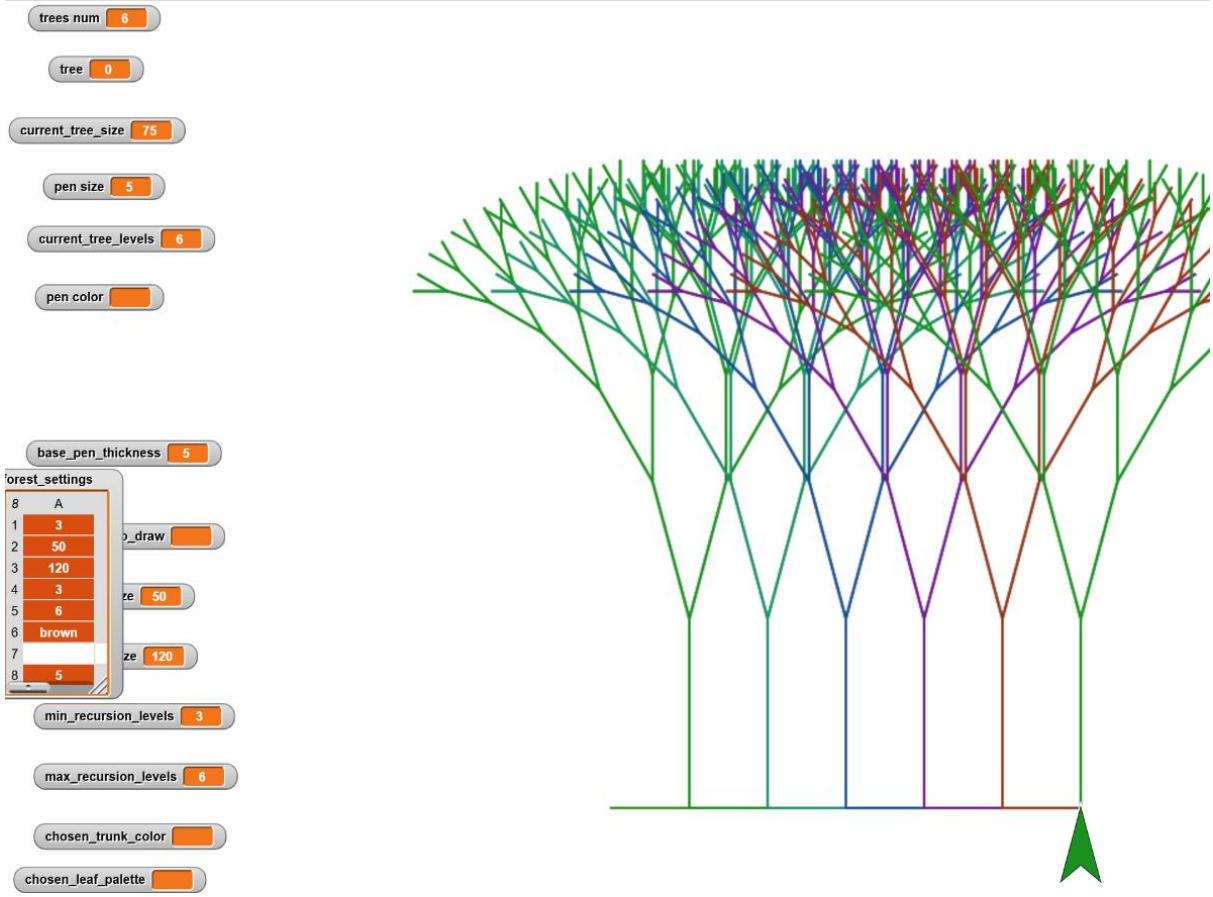
    // --- Move back down the current segment ---
    move ((0) - (current_size)) steps

  end // End of the main "if <(levels) > (0)> then"
 end // End of the "define Draw_Recursive_Tree" block
```

Due that extensions aren’t working so we will make more than tress with more simplified program.

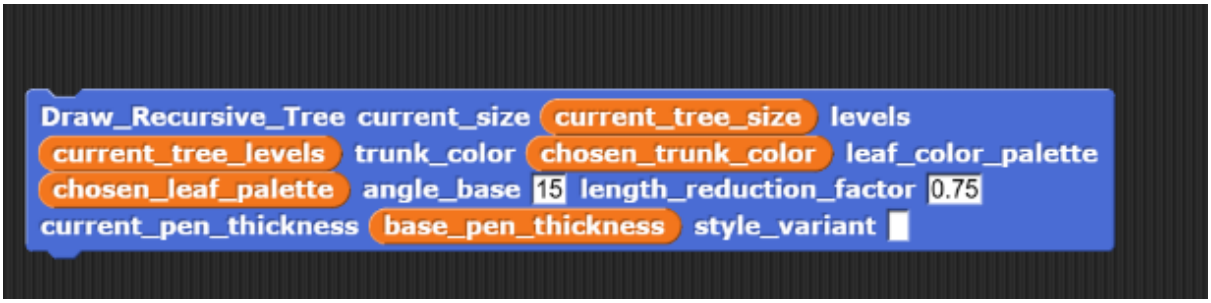


3 trees



6 trees with random size of 75 and with 6 levels for each.

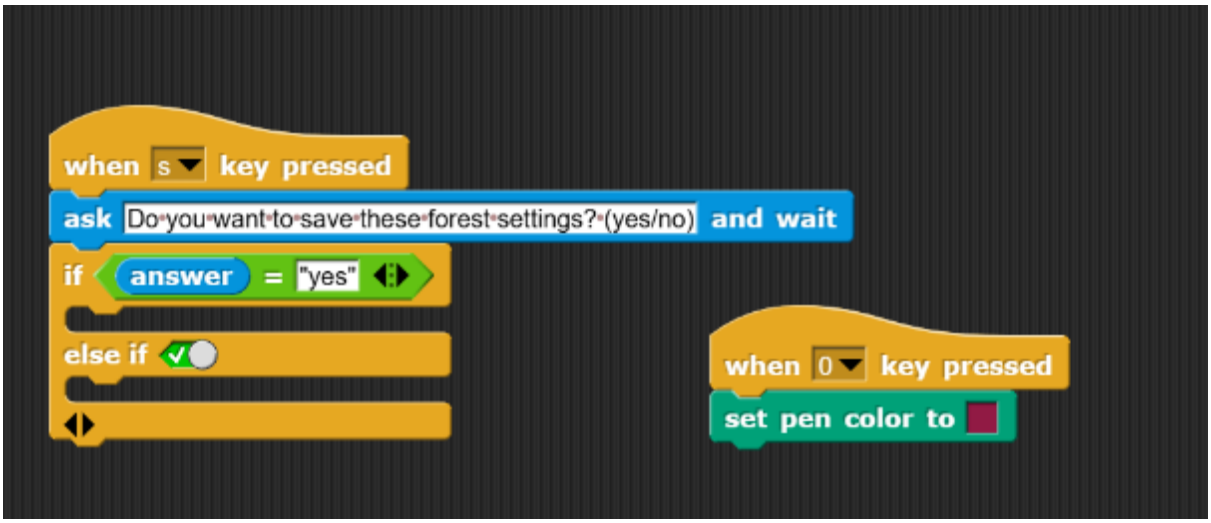
The old algorithm needs some improvement to work which generates tree with colors of each branch or trunk and the number of them based on the user input.



Function to convert from string to number.



You will need the extension of the lists to use the repeat with index block.



Using extensions I would make “S” as save button and “0” button to set the pen color to the desired color.